

Meeting 01 面试参考答案

1. 介绍一下租客端智能找房助手项目

AptGuide 3.0 是我独立开发的一个面向租客的智能找房助手，支持自然语言找房、租赁规则咨询、预约看房和租约查询四个核心场景。技术栈是 Python 和 FastAPI 做后端，RAG 检索用 Milvus 向量数据库，状态管理用 MySQL 加 Redis，前端是 Vue3。

这个项目的核心设计原则是 LLM-first，也就是所有用户输入都先经过大模型做结构化理解，输出任务类型、硬性条件、软偏好、检索查询和风险等级，而不是用关键词匹配。然后根据理解结果分发到七个不同的业务流程。

在可靠性方面，写操作比如预约看房采用待确认动作模式，模型只负责识别意图，系统先生成确认卡片，用户确认后才由业务后端执行真实写入。房源推荐也做了业务事实校验，向量检索返回的房源必须通过后端验证才能标为已验证。

2. 智能找房的流程是怎样的

以找房为例，完整流程是这样的：

第一步是安全拦截，先检查用户消息是否包含隐私敏感词，比如室友手机号、身份证这些，命中就直接拦截。

第二步是待确认动作检查，如果 session 中有待确认动作，比如之前预约没确认，优先处理确认或拒绝。

第三步是 LLM 结构化理解，调用大模型解析用户输入，输出一个结构化结果，包含路由、任务类型、硬性条件、软偏好、检索查询和风险等级。

第四步是校验与澄清，如果置信度低于阈值或者字段校验失败，自动转为澄清流程。

第五步是根据路由分发到找房流程。

第六步是 RAG 检索管线，包括多查询向量召回、房源身份解析、业务后端验证、LLM 偏好打分、多维加权排序。

最后组装房源卡片返回给前端。整个过程中模型负责理解和检索规划，系统负责业务事实校验和安全控制。

3. 为什么用大模型解析硬性偏好，而不是规则引擎

因为用户表达方式太多了。比如用户说天河附近两千以内的单间，需要同时提取区域、租金上限、房型三个硬性条件。

如果用规则引擎，每新增一种表达方式就要加正则或关键词规则，比如天河、天和、天河区、珠江新城那边都要覆盖，维护成本很高。

用大模型的话，不管用户怎么表达，都能输出结构化的硬性条件字段。代码里还有实体解析模块做确定性后处理，比如天河映射到天河区、单间映射到标准房型枚举，保证最终字段值是标准的。

所以核心思路是大模型做语义理解，代码做字段标准化和校验，两者配合。

4. 用户偏好字段是如何解析的

分两步。

第一步是 LLM 结构化提取。通过 system prompt 告诉模型输出标准格式的结构化结果，其中硬性条件字典包含租金、区域、房型、付租方式等标准字段。prompt 里给了十个 few-shot 覆盖不同场景，包括找房、问答、预约、租约、模糊输入。

第二步是确定性实体解析。LLM 输出的自然语言值经过一个别名映射表转换为标准值。比如区域有十一个别名映射，天河到天河区、番禺到番禺区。房型有十四个别名映射，单间到标准房型枚举。付租方式有十个别名映射，月付到标准枚举。

如果无法解析，会标记为未解析字段，后续可能触发澄清。这样做的好处是大模型处理语义多样性，代码保证字段标准化。

5. 字段校验和兜底分别由大模型还是后端完成

字段校验完全是后端代码完成的，不依赖大模型。具体来说：

白名单校验，硬性条件只允许八个标准字段，多余的直接丢弃。枚举校验，付租方式只允许月付、季付、半年付、年付四种；房型只允许单间、一房、两房、合租、整租几种。形状校验，比如路由是找房时任务类型必须是房间搜索或知识问答，否则视为非法。置信度校验，低于阈值自动转澄清。

兜底也是后端完成的。如果校验失败或 LLM 调用异常，会返回一个确定性的澄清结果，包含默认的中文澄清话术，不依赖大模型生成。

这样设计的原因是校验和兜底是安全边界，必须是确定性的，不能依赖概率性的大模型。

6. 后端代码在哪些环节做了校验

后端在多个环节做了校验，形成纵深防御。

第一层是安全拦截层，检查隐私敏感词，命中直接拦截，在 LLM 调用之前。

第二层是理解校验层，校验 LLM 输出的路由、任务、动作是否合法，硬性条件是否在白名单内，枚举值是否有效，置信度是否达标。

第三层是实体解析层，将 LLM 输出的自然语言值映射为标准枚举，无法解析的标记为未解析。

第四层是 RAG 校验层，房源必须通过业务后端验证才能标为已验证，知识库规则必须有文档 ID、审核状态、审核人。

第五层是写操作确认层，预约等写操作必须用户确认后才执行，三十分钟过期。

第六层是数据隔离层，系统不直接访问业务数据库，所有业务读写通过内部 HTTP 接口。

每一层都是确定性代码，不依赖大模型。

7. 自然语言查询数据库时，是否会直接生成 SQL

这要看是哪个项目。

AptInsight 也就是运营分析助手会直接生成 SQL。流程是意图识别，然后 LLM 生成 SQL，再经过 SQL Guard 安全校验，最后只读执行。但有严格的安全边界：SQL Guard 基于 AST 做校验，只允许 SELECT，拦截多语句、危险操作、敏感字段；有表白名单和字段白名单；用只读数据库账号；查询超时十秒，最多返回两百行。

AptGuide 3.0 也就是找房助手不会生成 SQL。它通过内部 HTTP 接口查询业务数据，不直接访问数据库。设计原则是系统只管运行时状态，业务事实由业务系统拥有。

8. 这个项目是 workflow 还是 agent？记忆层怎么设计

AptGuide 3.0 更接近 workflow，但有 agent 的特征。

严格来说，它是一个结构化的流程调度系统。LLM 理解用户意图后，分发到七个预定义的业务流程，每个流程内部是固定的管线步骤。这不是典型的 agent loop，也就是模型自主决定下一步，而是模型做一次理解，代码做分发和执行。

选择这种设计的原因是 C 端场景需要可控性和可预测性，不能让模型自主决定是否执行写操作。

记忆层设计方面：短期记忆用 MySQL 加 Redis 存储会话状态，包含上一轮任务、元数据、待确认动作，二十四小时过期。长期偏好存 MySQL，支持增删改查。待确认动作 Redis 存储，三十分过期，用于预约等写操作的确认流程。每轮对话的用户和助手消息都存 MySQL，支持回溯。

9. workflow 的整体流程是什么？哪些节点用了 agent

以 AptInsight 为例，整体流程是：用户问题先进入意图识别节点，然后根据意图走不同分支。如果是业务分析问题，走 SQL 生成、安全校验、只读执行、图表构建、答案生成的链路。如果是闲聊或超范围，直接走答案生成。

六个节点中，三个用了 LLM：意图识别用小模型做分类，SQL 生成用专业模型做自然语言转 SQL，答案生成用小模型做总结。另外三个是纯代码：安全校验用 sqlglot 做 AST 检查，只读执行跑数据库查询加数据脱敏，图表构建生成 ECharts 配置。

AptGuide 3.0 没有用 LangGraph，而是自己实现了流程调度。只有理解阶段用 LLM，七个业务流程内部主要是确定性代码加 RAG 管线。

10. 业务流程中哪些节点真正属于 agent

严格来说，两个项目中真正有 agent 特征的节点很少。

AptInsight 中，SQL 生成节点是最接近 agent 的，模型需要理解语义、选择表和字段、构造查询。但它是单步转换，没有 tool calling，没有循环，不算真正的 agent。

AptGuide 3.0 中，理解节点一次调用同时完成意图识别、字段提取、风险评估、检索查询生成，这是多维度语义理解，比单纯的分类更有 agent 特征。

但整体来说，两个项目都是 workflow 主导，模型做理解和生成，代码做分发和执行。这不是缺陷，而是设计选择，C 端和 B 端场景需要可控性。

11. 如果只是调用大模型 API，为什么能称为 agent？agent loop 体现在哪里

严格来说，我的项目不是典型的 agent loop 架构。典型 agent loop 是模型自主决定下一步动作、调用工具、观察结果、再决定下一步，也就是 ReAct 模式。

我的项目更准确的定义是 LLM 驱动的工作流，模型做一次理解或生成，代码做分发和执行。

但之所以在简历中写 AI Agent 项目，是因为：第一，LLM 是决策核心，意图识别、字段提取、风险评估、SQL 生成都由 LLM 完成，不是规则引擎。第二，多步骤管线，找房流程有七个步骤，每一步都可能影响最终结果。第三，动态路由，根据 LLM 理解结果动态分发到不同流程。

如果面试官追问，我会承认这不是 agent loop，而是结构化流程调度加 LLM 理解。这种设计在 C 端场景下更可控、更可预测。

12. 意图识别节点会输出哪些内容

以 AptInsight 为例，意图识别节点输出意图分类和原因。意图有三个值：业务分析问题需要查数据库，闲聊就是问候和系统能力问题，超范围包括隐私查询、写操作、无关话题。

如果 JSON 解析失败，会用关键词兜底，比如包含你好就判为闲聊。

AptGuide 3.0 的理解节点输出更丰富，一次调用输出：路由类型有八种，任务类型有九种，领域有十一种，动作有十四种，硬性条件字典包含租金、区域、房型等，软偏好列表，一到四条检索查询，风险等级和响应模式，置信度，以及是否需要澄清。

13. SQL 取数节点是 agent 还是 workflow

SQL 生成节点本质上是 workflow 中的一个 LLM 调用节点，不是独立的 agent。

它的流程是固定的：注入 Schema 上下文，也就是二十九张表的结构，加上指标口径和十一个 few-shot 示例，然后调用 LLM 生成 SQL，解析响应提取 SQL，最后做基础安全校验。

这不是 agent，因为没有 tool calling，模型不能调用外部工具；没有循环，一次调用就结束；没有自主决策，输入输出格式固定。

选择 workflow 而不是 agent 的原因是 Text-to-SQL 任务本身是单步转换，不需要多轮交互。模型的职责是理解问题然后生成 SQL，安全校验和执行由代码控制。

14. SQL 生成和执行的安全边界是什么

SQL 安全是四层纵深防御。

第一层是 Prompt 约束，明确要求只生成 SELECT，不能包含写操作，要求使用白名单表和字段，要求加逻辑删除条件。

第二层是代码预校验，在 SQL 生成节点内用正则检查 SELECT-only，拦截危险关键词，检查分号防止多语句。

第三层是 AST 级安全校验，用 sqlglot 解析 SQL 为 AST，只允许 SELECT 语句，拦截多语句，做表白名单校验，做字段白名单校验，拦截敏感字段，递归校验子查询。

第四层是执行层保护，用只读数据库账号，查询超时十秒，最多返回两百行，执行后对敏感数据脱敏。

任何一层失败都会拦截，返回用户友好的错误信息。安全类错误不会重试，只有格式类错误才重试一次。

15. 会向模型提供哪些数据库表字段信息

在知识文档中提供完整的 Schema，包含二十九张表的完整定义，每张表包括表名和中文说明、所有字段名和类型和中文说明、字段间的关联关系、安全标注，也就是哪些是敏感字段、哪些是完全禁止的字段。

核心业务表包括公寓信息、房间信息、预约看房、租约、租户评价、浏览记录。辅助表包括地理信息表、字典表、关联表。

另外还提供业务指标口径文档，比如预约量、完成率、转化率的 SQL 公式。还有十一个完整的自然语言转 SQL 示例。

这些上下文在启动时加载一次，后续复用。

16. 生成 SQL 后如何保障安全

生成 SQL 后必须经过 SQL Guard 才能执行。SQL Guard 基于 sqlglot 做 AST 级安全校验，具体检查包括：

单语句检查，用 parse_one 隐式检查加 parse 显式检查语句数量。SELECT-only 检查，验证 AST 类型。表白名单检查，遍历 AST 中所有表名，每个必须在白名单中。字段白名单检查，遍历所有字段，检查是否允许访问、是否被禁止。子查询递归检查。

校验通过后返回重写后的 SQL，也就是 sqlglot 标准化后的版本。后续执行节点只执行重写后的 SQL，不执行原始生成的 SQL。

如果校验失败，根据违规类型生成用户友好的错误信息和修复建议，不会执行 SQL。

17. 数据库账号、token、连接信息等敏感信息如何处理

配置层方面，所有敏感配置使用 Pydantic 的 SecretStr 类型，打印时自动显示为星号。通过环境变量管理，不硬编码在代码中。诊断报告生成时会自动脱敏敏感字段。

运行时方面，AptInsight 使用只读数据库账号，即使 SQL 绕过了 Guard 也无法执行写操作。AptGuide 3.0 不直接访问数据库，所有业务操作通过内部 HTTP 接口，接口需要 Token 认证。LangSmith tracing 中的敏感信息也会被脱敏。

日志层方面，SQL 日志截断到五百字符，不记录完整的 API Key 或 Token。

18. 账号、密码、token 等敏感数据有没有脱敏

有，多层脱敏。

配置脱敏方面，Pydantic SecretStr 类型序列化时自动掩码。

诊断报告脱敏方面，会扫描所有字段，命中 API Key、Token、密码等关键词的自动替换为已脱敏。

查询结果脱敏方面，AptInsight 对查询结果做后处理：手机号显示为一三八星五六七八，姓名显示为张星星，地址保留省市区详细地址掩码，身份证号显示为已拦截。字段名匹配用精确集合，不用子串匹配，避免公寓名称被误脱敏。

KB 内容校验方面，会检测知识库规则中的个人信息，比如手机号、身份证号、银行卡号，命中则标记。

19. 为什么内部调用场景下你认为不需要脱敏

不是不需要脱敏，而是脱敏的策略不同。

AptInsight 面向的是运营人员，也就是 B 端用户。运营人员本身就有查看业务数据的权限，所以查询结果中的手机号、姓名会做掩码脱敏，但不会完全隐藏，因为运营人员需要这些信息做业务决策。

而 AptGuide 3.0 面向的是租客，也就是 C 端用户，脱敏更严格。租客只能看到自己的租约信息，不能查询其他租户的手机号、身份证等隐私信息。安全拦截层会在 LLM 调用之前就拦截隐私查询请求。

所以脱敏策略是按角色分级的，不是内部就不脱敏。

20. 有没有 human-in-the-loop 来确认 SQL 执行

AptInsight 没有做人工确认 SQL 执行。原因是：目标用户是运营人员，他们期望快速得到分析结果，每次确认会降低效率。安全靠技术手段保证，SQL Guard 加只读账号加查询超时加行数限制已经足够。业务场景是只读分析，不涉及数据修改。

但我在 AptGuide 3.0 中做了 human-in-the-loop，不过不是确认 SQL，而是确认写操作。预约看房时，LLM 识别意图后，系统先生成确认卡片，用户确认后才调用接口创建预约，三十分钟过期。如果面试官问为什么 AptInsight 不做，可以回答安全策略的选择取决于场景，只读分析用技术手段，写操作用确认模式。

21. 没有人工确认时，如何保证 SQL 执行安全

通过四层纵深防御，核心是：

第一层 Prompt 约束告诉模型只生成 SELECT。第二层代码预校验用正则拦截危险关键词。第三层 AST 安全校验用 sqlglot 做表白名单加字段白名单加敏感字段拦截。第四层执行层保护用只读账号加超时十秒加最多两百行加数据脱敏。

关键设计是只读数据库账号。即使前三层都被绕过，理论上不可能，数据库账号没有写权限，SQL 执行会报权限错误。

另外 SQL Guard 失败后不会重试。安全类错误说明 prompt 约束没有生效，重试大概率还是错的，反而增加延迟。只有格式类错误才重试一次。

22. 有没有做 SQL 边界校验，防止大查询拖垮数据库

有，三层保护。

第一层是查询超时，MySQL 级别设置十秒超时，超过自动中断。第二层是行数限制，最多返回两百行，不会把整张表拉回来。第三层是只读账号，不能执行创建索引、修改表结构等操作。

另外 SQL Guard 会拦截没有 WHERE 条件的全表扫描，因为表有逻辑删除字段，prompt 要求必须加这个条件。

如果面试官问为什么不加更复杂的限制，可以回答当前是 MVP 阶段，运营人员使用频率不高，十秒超时加两百行限制已经够用。后续可以加查询复杂度评估、慢查询日志告警等。

23. agent 或 workflow 中有多少个工具

AptInsight 有六个节点，不算工具调用，是 workflow 节点：意图识别、SQL 生成、安全校验、只读执行、图表构建、答案生成。其中三个用 LLM，三个是纯代码。

AptGuide 3.0 有七个业务流程，相当于工具：澄清、找房、知识问答、预约看房、租约查询、偏好记忆、人工转接。

另外还有一个外部工具接口，通过 HTTP 调用业务后端的五个接口：获取单个房间、搜索验证房间、创建预约、列出我的预约、列出我的租约。

如果面试官问为什么不多加工具，可以回答当前场景覆盖了找房、咨询、预约、租约四个核心需求，工具数量够用。工具太多会增加理解和分发的复杂度。

24. 整个 workflow 中有多少个工具函数

如果把 AptInsight 的每个节点看作一个工具函数，六个：意图识别、SQL 生成、安全校验、只读执行、图表构建、答案生成。

如果算上辅助函数：SQL 安全校验、数据库执行、数据脱敏、失败分类、提取表名等。核心就是六个节点函数，辅助函数是被节点调用的。

AptGuide 3.0 的话，七个业务流程加一个理解模块加一个安全拦截，加上 RAG 管线中的多个辅助函数，包括向量召回、排序、打分、置信度检查等。

25. LangGraph 在项目中是如何体现的

在 AptInsight 中，LangGraph 体现在五个方面。

第一是 StateGraph 定义，用 StateGraph 定义六个节点。第二是条件路由，用 add_conditional_edges 定义动态分支，比如意图识别后根据意图走不同路径。第三是共享 State，所有节点读写同一个 TypedDict，包含问题、意图、SQL、查询结果、回答等字段。第四是编译和执行，调用 compile 编译图，然后 ainvoke 异步执行。第五是可观测性，集成 LangSmith tracing，每次运行都有完整的 trace 记录。

AptGuide 3.0 没有用 LangGraph，而是自己实现了流程调度，因为它的流程更简单，一次理解、一次分发、一次执行，不需要 LangGraph 的图编排能力。

26. LangGraph 中有没有动态路由

有意图识别节点有动态路由。路由函数根据 state 中的意图值动态决定下一步：业务分析走 SQL 生成链路，闲聊直接写回答，超范围直接写回答，错误也直接写回答。

安全校验节点也有动态路由：SQL 安全就执行，不安全就直接写错误回答。

只读执行节点也有动态路由：执行成功走图表构建，执行失败走答案生成。

所以整个 workflow 有三个动态路由点，不是纯线性的。

27. 节点和路由很少时，为什么还要选择 LangGraph

选择 LangGraph 的原因有四个。

第一是共享 State 管理。LangGraph 的 StateGraph 天然支持所有节点共享一个数据结构，不需要手动传递参数。六个节点都需要读写问题、意图、SQL、结果、回答等字段，用 StateGraph 很自然。

第二是条件路由的声明式写法。比 if-else 链更清晰，路由逻辑和节点逻辑分离。

第三是可观测性。LangGraph 集成 LangSmith，每次运行自动记录 trace，包括每个节点的输入输出、耗时、LLM 调用详情。这对调试和评测很重要。

第四是可扩展性。后续如果要加重试循环，比如安全校验失败后重新生成 SQL，或者并行节点，LangGraph 天然支持。

如果节点更少，比如只有两三个，确实不需要 LangGraph，直接用函数调用就够了。但六个节点加三个动态路由加共享 State，LangGraph 是合理的选择。

28. 你怎么理解 Harness

我理解的 Harness 是 Agent 的测试和评测框架，类似于软件工程中的测试套件。

在我的 AptInsight 项目中，Agent Eval Harness 包括测试数据集，四十个 YAML 格式的测试用例，覆盖业务分析、安全测试、边界场景、复杂查询、拒答质量。每个用例有校验规则，比如应该拒绝、SQL 中不能包含的关键词、必须使用的表、回答中必须包含的关键词、图表类型。

还有评测逻辑，自动运行所有用例，校验结果。评测报告自动统计通过率、安全通过率、核心指标通过率、平均耗时。多模型对比，支持不同模型在各环节的表现对比。

Harness 的价值是让 Agent 的质量可量化、可回归、可对比。每次改 prompt 或换模型，跑一遍 Harness 就知道影响了什么。

29. prompt engineering、context engineering 和 harness engineering 有什么区别

Prompt Engineering 是设计给大模型的指令，让它理解任务、输出格式、约束条件。比如我的 SQL 生成 prompt 包含角色定义、安全规则、few-shot 示例。

Context Engineering 是设计给大模型的上下文信息，让它有足够的背景知识完成任务。比如我注入 Schema 文档、指标口径、few-shot 示例。Context 的质量直接决定模型输出的质量。

Harness Engineering 是设计测试和评测框架，量化 Agent 的质量。比如我的四十个测试用例、校验规则、评测报告、多模型对比。Harness 让 Agent 好不好从主观判断变成客观数据。

三者的关系是：Prompt 告诉模型做什么，Context 告诉模型用什么信息做，Harness 告诉我们做得好不好。在实际项目中三者是迭代的，改 prompt 跑 harness 看结果调 context 再跑 harness。

30. 自然语言查询、预约、租约、房源等工具是如何召回的

AptGuide 3.0 的工具召回不是传统的工具选择，而是基于 LLM 理解的确定性分发。

LLM 理解用户输入，输出路由和任务类型，然后流程调度器根据路由做确定性分发：找房路由分发到找房流程，问答路由分发到知识问答流程，预约路由分发到预约流程，租约路由分发到租约流程，记忆路由分发到偏好记忆流程，人工路由分发到人工转接流程，澄清路由分发到澄清流程。

这比传统的让模型选择工具调用更可控。模型只需要做好一件事，理解用户意图并输出结构化结果。分发逻辑由代码控制，不会出现模型调错工具的情况。

RAG 检索中的房源召回是另一条路径，多查询向量召回、身份解析、业务验证、偏好打分、多维排序。

31. 系统是否支持多轮对话

支持，但不是传统的上下文拼接式多轮对话。

AptGuide 3.0 的多轮对话通过 session 加待确认动作实现。

场景一，预约确认流程。用户说想预约明天看房，系统生成确认卡片和待确认动作。用户说确认，系统从 session 中加载待确认动作，执行预约。

场景二，偏好记忆。用户说想要天河区的单间，系统记录偏好到长期记忆。下次对话用户说帮我找房，系统从记忆中加载偏好，注入到检索条件。

场景三，澄清后继续。用户说帮我找房，系统问您的预算和区域是。用户说天河两千以内，系统结合上一轮的意图继续执行。

Session 数据存储存储在 MySQL 加 Redis，二十四小时过期。每轮对话的消息都持久化。

32. 如果支持多轮对话，如何保持长记忆

短期记忆方面，存储在 MySQL 加 Redis，二十四小时过期，包含上一轮任务、元数据、待确认动作。每轮对话加载 session，处理，更新 session。

长期偏好方面，存储在 MySQL，永久保存，支持增删改查，最多返回十条偏好。

消息历史方面，每轮对话的用户和助手消息都存 MySQL，包含会话 ID、用户 ID、请求 ID、角色、内容、元数据，支持回溯和审计。

Trace 方面，每次请求的完整 trace 存 MySQL，包含理解结果、检索结果、排序结果、最终回答，支持 LangSmith 可视化。

当前没有做对话上下文拼接，也就是不会把前几轮消息塞进 LLM prompt，而是通过 session 状态和 memory 偏好来保持连贯性。这是一个设计选择，减少 token 消耗，提高响应速度。

33. 用户想基于查询结果继续对话时，系统怎么处理

当前的设计是每轮独立理解，不依赖前几轮的查询结果。

比如用户问天河区有什么房，返回五个房源卡片，用户说第二个怎么样，系统会重新理解第二个怎么样，但因为没有前一轮的结果上下文，可能会触发澄清。

这是一个已知的局限性。如果要支持基于结果继续对话，需要将前一轮的查询结果摘要存入 session，在下一轮理解时将摘要注入上下文，或者做指代消解。

目前的替代方案是：用户可以基于偏好继续对话，比如我之前说过想要天河的。用户可以基于待确认动作继续对话，比如确认预约。但不能直接基于查询结果做追问。

如果面试官问，可以承认这个局限性，并说明改进方向。

34. 缓存项目是否涉及并发

SmartLease 项目中 Redis 缓存的使用场景：

房间详情缓存是读多写少的场景。读的时候先查 Redis，命中直接返回。写的时候缓存 miss 时查数据库，结果写入 Redis。失效策略是管理员更新房间信息时删除缓存 key。

验证码缓存是 TTL 管理场景。短信验证码存 Redis，十分钟过期。六十秒内不允许重复发送，检查 key 是否存在。

登录 Token 管理方面，JWT Token 存 Redis，支持主动失效。

并发场景方面，多个用户同时查询同一房间，Redis 缓存命中，不走数据库。管理员更新房间加用户同时查询，可能读到旧数据。当前用的是删除缓存策略，更新时删 key，下次读时重建。

如果面试官问并发问题，可以回答当前是单机部署，QPS 不高，删除缓存加重建的策略够用。如果要支持更高并发，可以考虑分布式锁或延迟双删。

35. 为什么要做 Redis 缓存

SmartLease 项目做 Redis 缓存的原因有三个。

第一是热点数据加速。房间详情是高频查询，租客浏览房源，每次查数据库要关联多张表，缓存后直接返回，减少数据库压力。

第二是会话管理。验证码、Token 等需要 TTL 支持的数据，Redis 天然支持过期时间，比数据库方便。

第三是读多写少的场景适合缓存。房间信息更新频率低，管理员操作，但查询频率高，租客浏览，缓存命中率高。

具体的缓存策略是 Key 格式为房间 ID，Value 是序列化对象，没有固定 TTL，靠主动删除，管理员更新时删 key。读取时先查 Redis，miss 了查数据库，写入 Redis，返回结果。

如果面试官问为什么不用本地缓存，可以回答本地缓存无法跨实例共享，多实例部署时会有一致性问题。Redis 是集中式缓存，所有实例共享。

36. 缓存雪崩是否需要 Redis 集群？你是否部署了集群

缓存雪崩是指大量缓存同时过期，导致请求全部打到数据库。

是否需要 Redis 集群取决于场景。如果是高并发场景，比如秒杀、热门商品，需要集群保证可用性和吞吐量。如果是普通业务场景，比如房间查询，单机 Redis 加合理的过期策略就够了。

我的 SmartLease 项目没有部署 Redis 集群，因为这是一个学习项目，QPS 不高，单机 Redis 足够支撑当前负载。缓存雪崩的预防靠随机化 TTL，给不同 key 加随机偏移量，不靠集群。

如果要部署集群，可以考虑 Redis Cluster 自动分片支持高可用，Redis Sentinel 主从切换保证可用性，或者本地缓存加 Redis 二级缓存减少 Redis 压力。

37. 如果 Redis 宕机，是否会导致缓存雪崩

Redis 宕机不等于缓存雪崩，但会导致缓存穿透，也就是所有请求直接打到数据库。

我的项目处理方式是：房间详情查询方面，Redis 宕机就缓存 miss，查数据库，返回结果，功能正常只是慢一点。验证码和 Token 方面，Redis 宕机就无法存储和读取，登录功能不可用，这是更严重的问题。

如果要保证高可用，可以做多级缓存，本地缓存加 Redis，Redis 宕机时降级到本地缓存。熔断降级，检测到 Redis 不可用时直接查数据库，不等待超时。Redis 高可用，主从加 Sentinel 或 Cluster，自动故障转移。限流，数据库前面加限流，防止 Redis 宕机时数据库被打爆。

当前项目是学习项目，没有做这些高可用措施。如果面试官问，可以说明改进方向。